

Lista de Exercícios - Threads

Programação Orientada a Objetos

Exercício 1

A diferença principal está na forma de criar a thread.

Na transparência 5, ThreadSimples herda de Thread, então o próprio objeto já é uma thread. Por isso, ela pode ser iniciada direto com `new ThreadSimples().start()`.

Na transparência 8, ThreadSimples implementa Runnable. Nesse caso, o objeto representa só a tarefa, então é preciso criar uma Thread para executá-lo: `new Thread(new ThreadSimples()).start()`.

Outra diferença é a chamada de sleep. No primeiro programa aparece `sleep(1000)` porque a classe herda de Thread. No segundo aparece `Thread.sleep(1000)` porque a classe implementa Runnable e não herda esse método diretamente.

No geral, os dois programas fazem a mesma ideia, mas o segundo é mais flexível, porque usando Runnable a classe ainda pode herdar de outra classe.

Exercício 2

A lógica foi dividir a matriz em quatro quadrantes. O valor `meio = n / 2` separa linhas e colunas em duas partes, e cada thread fica responsável por somar um quadrante diferente. No final, o `join()` garante que a matriz resultado só seja impressa depois que as quatro threads terminarem.

```
import java.util.Scanner;

class SomaQuadrante extends Thread {
    private int[][] a;
    private int[][] b;
    private int[][] c;
    private int linhaInicio;
    private int linhaFim;
    private int colunaInicio;
    private int colunaFim;

    SomaQuadrante(
        int[][] a,
        int[][] b,
        int[][] c,
        int linhaInicio,
        int linhaFim,
        int colunaInicio,
        int colunaFim
    ) {
        this.a = a;
        this.b = b;
        this.c = c;
        this.linhaInicio = linhaInicio;
        this.linhaFim = linhaFim;
        this.colunaInicio = colunaInicio;
        this.colunaFim = colunaFim;
    }

    public void run() {
        for (int i = this.linhaInicio; i < this.linhaFim; i++) {
            for (int j = this.colunaInicio; j < this.colunaFim; j++) {
                this.c[i][j] = this.a[i][j] + this.b[i][j];
            }
        }
    }
}

public class Main {
    static void lerMatriz(Scanner scanner, int[][] matriz) {
        for (int i = 0; i < matriz.length; i++) {
            for (int j = 0; j < matriz.length; j++) {
                matriz[i][j] = scanner.nextInt();
            }
        }
    }
}
```

```

    }
}

static void imprimirMatriz(int[][] matriz) {
    for (int[] linha : matriz) {
        for (int valor : linha) {
            System.out.print(valor + " ");
        }
        System.out.println();
    }
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    System.out.print("N: ");
    int n = scanner.nextInt();

    int[][] a = new int[n][n];
    int[][] b = new int[n][n];
    int[][] c = new int[n][n];

    System.out.println("Matriz A:");
    lerMatriz(scanner, a);

    System.out.println("Matriz B:");
    lerMatriz(scanner, b);

    int meio = n / 2;

    Thread[] threads = {
        new SomaQuadrante(a, b, c, 0, meio, 0, meio),
        new SomaQuadrante(a, b, c, 0, meio, meio, n),
        new SomaQuadrante(a, b, c, meio, n, 0, meio),
        new SomaQuadrante(a, b, c, meio, n, meio, n)
    };

    for (Thread thread : threads) {
        thread.start();
    }

    try {
        for (Thread thread : threads) {
            thread.join();
        }
    } catch (InterruptedException e) {
    }

    System.out.println("Matriz soma:");
    imprimirMatriz(c);
}
}

```

Exercício 3

O `SwingUtilities.invokeLater` é usado para mandar a atualização da interface para a EDT, que é a thread do Swing responsável pelos componentes gráficos.

Nesse programa, o `run()` está sendo executado pela thread criada em `new Thread(this).start()`. Como essa thread não é a EDT, ela não deve atualizar a `JProgressBar` direto. Por isso o `invokeLater` é usado.

Exercício 4

Não precisa, porque essa chamada já está dentro do `Runnable` que foi passado para o `invokeLater`. Então, quando `pBar.setValue(cont)` é executado, ele já está rodando na EDT.

Exercício 5

Não seria o certo. Se `pBar.setValue(cont++)` fosse executado direto no `run()` externo, a atualização da barra seria feita pela thread auxiliar, e não pela EDT. Como a `JProgressBar` é um componente Swing, o ideal é que a parte gráfica seja atualizada pela EDT.

Exercício 6

Sim, dá para trocar `invokeLater` por `invokeAndWait`. Como a chamada está sendo feita por uma thread de trabalho, isso funciona sem problema. A mudança principal é usar `invokeAndWait` e tratar também a exceção `InvocationTargetException`.

```
import javax.swing.*;
import java.lang.reflect.InvocationTargetException;

public class TestaThread extends JFrame implements Runnable {
    JProgressBar pBar = new JProgressBar(JProgressBar.HORIZONTAL, 0, 10);
    int cont = 0;

    TestaThread() {
        super("Thread com Swing");
        add(pBar);
        pack();
        setVisible(true);
        new Thread(this).start();
    }

    public void run() {
        try {
            while (cont <= 10) {
                SwingUtilities.invokeLaterAndWait(
                    new Runnable() {
                        public void run() {
                            pBar.setValue(cont++);
                        }
                    }
                );
                Thread.sleep(1000);
            }
        } catch (InterruptedException e) {
        } catch (InvocationTargetException e) {
        }
    }

    static public void main(String[] s) {
        new TestaThread();
    }
}
```

A diferença é a seguinte: `invokeLater` só agenda a tarefa e a thread continua a execução normalmente. Já o `invokeAndWait` agenda a tarefa e espera ela terminar. Ou seja, ele bloqueia a thread que fez a chamada até a EDT executar o bloco.

Por isso, `invokeAndWait` não deve ser chamado de dentro da própria EDT.

Exercício 7

Cada JTextField gira em uma thread separada. Depois, uma thread de controle espera as três terminarem com join(). As mudanças visuais são feitas com SwingUtilities.invokeLater, e o botão Jogar fica desabilitado enquanto o sorteio está acontecendo.

```
import javax.swing.*;
import java.awt.*;
import java.util.Random;

public class CacaNiquel extends JFrame {
    private JTextField[] campos;
    private JButton jogarButton;
    private int[] resultados;

    CacaNiquel() {
        this.campos = new JTextField[3];
        this.resultados = new int[3];

        this.setTitle("Caça-Níquel");
        this.setSize(360, 180);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        this.setLocationRelativeTo(null);
        this.setLayout(new BorderLayout(10, 10));

        JPanel numerosPanel = new JPanel(new GridLayout(1, 3, 10, 10));

        for (int i = 0; i < this.campos.length; i++) {
            this.campos[i] = new JTextField("1");
            this.campos[i].setEditable(false);
            this.campos[i].setHorizontalAlignment(JTextField.CENTER);
            this.campos[i].setFont(new Font("Arial", Font.BOLD, 28));
            numerosPanel.add(this.campos[i]);
        }

        this.jogarButton = new JButton("Jogar");
        this.jogarButton.addActionListener(e -> jogar());

        JPanel botaoPanel = new JPanel();
        botaoPanel.add(this.jogarButton);

        this.add(numerosPanel, BorderLayout.CENTER);
        this.add(botaoPanel, BorderLayout.SOUTH);
    }

    private void jogar() {
        this.jogarButton.setEnabled(false);

        new Thread(() -> {
            Thread[] threads = new Thread[3];

            for (int i = 0; i < threads.length; i++) {
                int indice = i;
                threads[i] = new Thread(() -> girar(indice));
                threads[i].start();
            }

            try {
                for (Thread thread : threads) {
                    thread.join();
                }
            } catch (InterruptedException e) {
            }

            SwingUtilities.invokeLater(() -> finalizarJogo());
        }).start();
    }

    private void girar(int indice) {
        Random random = new Random();
        int giros = random.nextInt(150) + 1;

        for (int i = 0; i < giros; i++) {
```

```
        int valor = random.nextInt(7) + 1;
        this.resultados[indice] = valor;

        SwingUtilities.invokeLater(() ->
            this.campos[indice].setText(String.valueOf(valor))
        );

        try {
            Thread.sleep(30);
        } catch (InterruptedException e) {
        }
    }
}

private void finalizarJogo() {
    boolean venceu =
        this.resultados[0] == this.resultados[1] &&
        this.resultados[1] == this.resultados[2];

    if (venceu) {
        JOptionPane.showMessageDialog(this, "Você venceu o jogo!");
    } else {
        JOptionPane.showMessageDialog(this, "Você perdeu o jogo.");
    }

    this.jogarButton.setEnabled(true);
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        CacaNiquel jogo = new CacaNiquel();
        jogo.setVisible(true);
    });
}
}
```